

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«Казанский национальный исследовательский технический
университет им. А.Н. Туполева – КАИ»**

**Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности**

Лабораторная работа № 2

Дисциплина: «Аппаратные средства телекоммуникационных
систем»

Тема: «Последовательный и параллельный алгоритмы
умножения больших матриц»

Работу выполнила
студент гр. 4404
Миронихина К.И.

Казань, 2026

Напишите программу перемножения больших матриц. Сравните время выполнения последовательной и параллельной программы на 2,4, 8 потоках (процессорах).

Код:

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <omp.h>
#include <windows.h>
using namespace std;

// Заполнение матрицы случайными числами
void fillMatrix(vector<vector<int>>& matrix, int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = rand() % 10;
        }
    }
}

int main() {
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);
    setlocale(LC_ALL, "Russian");
    srand(time(0));

    int rowsA, n, colsB;

    cout << "Введите число строк матрицы A: ";
    cin >> rowsA;
    cout << "Введите общее измерение (столбцы A = строки B): ";
    cin >> n;
    cout << "Введите число столбцов матрицы B: ";
    cin >> colsB;

    if (rowsA <= 0 || n <= 0 || colsB <= 0) {
        cout << "Ошибка: все размеры должны быть положительными!" << endl;
        return 1;
    }

    // Создание матриц
    vector<vector<int>> matrixA(rowsA, vector<int>(n));
    vector<vector<int>> matrixB(n, vector<int>(colsB));
    vector<vector<int>> matrixC(rowsA, vector<int>(colsB));

    fillMatrix(matrixA, rowsA, n);
    fillMatrix(matrixB, n, colsB);

    cout << "\n=== schedule(static) ===" << endl;

    // Перебор числа потоков
    for (int numThreads : {1, 2, 4, 8}) {
        omp_set_num_threads(numThreads);

        // Обнуление C перед каждым запуском
        for (int i = 0; i < rowsA; i++)
            for (int j = 0; j < colsB; j++)
                matrixC[i][j] = 0;

        double startTime = omp_get_wtime();

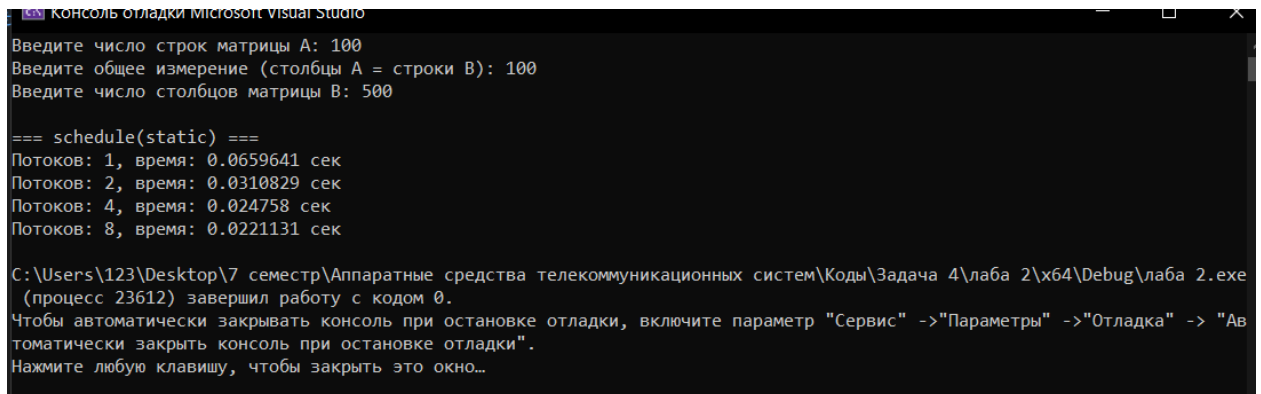
#pragma omp parallel for schedule(static)
        for (int i = 0; i < rowsA; i++) {
            for (int j = 0; j < colsB; j++) {
                int sum = 0;
                for (int k = 0; k < n; k++) {
                    sum += matrixA[i][k] * matrixB[k][j];
                }
                matrixC[i][j] = sum;
            }
        }

        double endTime = omp_get_wtime();

        cout << "Потоков: " << numThreads
              << ", время: " << endTime - startTime << " сек" << endl;
    }

    return 0;
}
```

Вывод:



```
Консоль отладки Microsoft Visual Studio
Введите число строк матрицы A: 100
Введите общее измерение (столбцы A = строки B): 100
Введите число столбцов матрицы B: 500

=== schedule(static) ===
Потоков: 1, время: 0.0659641 сек
Потоков: 2, время: 0.0310829 сек
Потоков: 4, время: 0.024758 сек
Потоков: 8, время: 0.0221131 сек

C:\Users\123\Desktop\7 семестр\Аппаратные средства телекоммуникационных систем\Коды\Задача 4\лаба 2\x64\Debug\лаба 2.exe
(процесс 23612) завершил работу с кодом 0.
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" -> "Параметры" -> "Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно...
```

Поскольку каждый поток работает со своими строками и пишет в свои ячейки результата, потоки не мешают друг другу и не требуют синхронизации внутри цикла; единственная синхронизация происходит в конце, когда все потоки завершают свои итерации и выстраиваются на неявном барьере, после чего выполнение продолжается последовательно. Ключ `schedule(static)` означает, что это деление итераций выполняется заранее и поровну, без обращений к общей «стопке задач» во время работы, что даёт минимум накладных расходов.